PYTHON INHERITANCE

INHERITANCE

- Inheritance is a process of creating a new class (derived class) from old class (base class).
- Constructing or Deriving **a new class from an existing class**.
- It is a pure object-oriented programming technique.
- New class : Child class / Sub class / Derived class
- Old class : Parent class / Super class / Base class

Characteristics of Inheritance

1. Like C++, in python one class can inherit more than one super class at a time but this is not possible in java/c#.
2. In python, there is **no interface concept** unlike java/c#
3. Sub class can access only the non-private members of the super class.

Super Class : Most general class (base class)

Sub Class : Most dependent class (derived class)

Construction of new class

- In python, new class can be derived using the syntax below

Syntax

```
class sub-class(base-class1, base-class 2, …n):
        # method definition
```

Where,

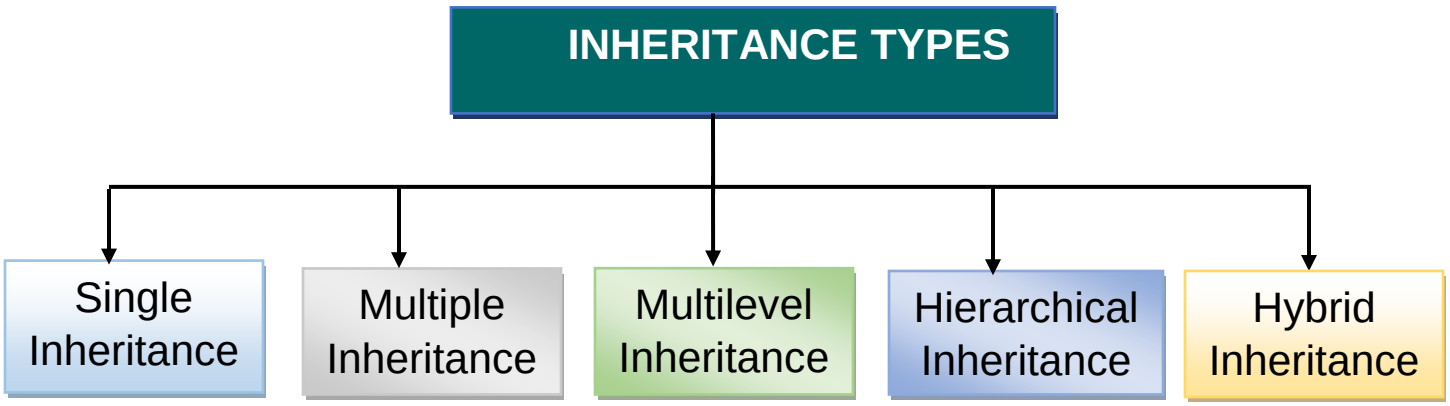
- The operator **()** is used for inheriting super classes
- Base-class 1, base-class 2, … base-class n are list of base classes
- Unlike java/c#, python derived class can inherit more than one base class at a time.

Example

```
# base class definition
class Vehicle:
    def disp():
        pass
# derived class definition
class Bus(Vehicle):
    pass
```

TYPES OF INHERITANCE

- In python, inheritance can be classified as five types. They are
  1. Single Inheritance
  2. Multiple Inheritance
  3. Multilevel Inheritance
  4. Hierarchical Inheritance
  5. Hybrid Inheritance



1. Single Inheritance

- Includes only one super class and sub class.

2. Multiple Inheritance

- Includes several super classes and one sub class.

3. Multilevel Inheritance

- Derived a new class from another sub class

4. Hierarchical Inheritance

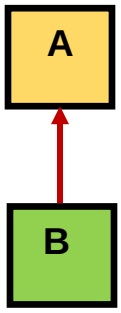
- Includes only one super class and several sub classes

5. Hybrid Inheritance

- Involving at least one form of inheritance.

1. SINGLE INHERITANCE

- Creating a new class (sub class) from only one super class is called as single inheritance.
- Like 1-1 mapping.



A ← Super Class

B ← Sub Class.

Figure 1.1 Single Inheritance

1. EXAMPLE OF SINGLE INHERITANCE

(single.py)

SOURCE CODE

```
# base class creation
class A:
# define method m1
    def m1(self):
        self.a=10
# derived class creation
class D(A):
# define derived class own method
    def m2(self):
        self.b=25
# define another method
    def sum(self):
        self.rs=self.a+self.b
        print("Number 1: ",self.a)
```

Public instance variable

Single Inheritance Creation

Public instance variable

```
        print("Number 2: ",self.b)
        print("Sum: ",self.rs)
# object creation for derived class
obj=D()
print("----------------------------------------")
print("\t Single Inheritance")
print("----------------------------------------")
# calling methods of both base and derived classes using derived class object
obj.m1()
obj.m2()
obj.sum()
print("----------------------------------------")
```

OUTPUT

```
TERMINAL    JUPYTER    PROBLEMS    OUTPUT    DEBUG CONSOLE

PS C:\Users\Ganesh\Documents\VSC Python Projects\inheritance> py single.py
----------------------------------------
         Single Inheritance
----------------------------------------
Number 1:  10
Number 2:  25
Sum:  35
----------------------------------------
PS C:\Users\Ganesh\Documents\VSC Python Projects\inheritance>
```

2. MULTIPLE INHERITANCE

- Creating a new class (sub class) from more than one super class is called as multiple inheritance.
- Like 1-many mapping.

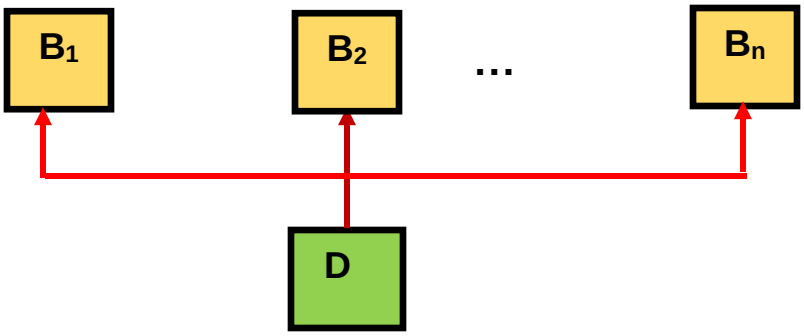


Figure 1.2 Multiple Inheritance

Where,

- **B_1, B_2, …B_n** are list of base classes
- **D** is a newly derived class.

2. EXAMPLE OF MULTIPLE INHERITANCE

(multiple.py)

SOURCE CODE

```
# base class 1
class Person:
    def m1(self):
        self.pname="Rohit"
        self.pid=14
    def p1(self):
        print("Name\t\t: ",self.pname)
        print("Id\t\t: ",self.pid)
# base class 2
class Personal:
    def m2(self):
        self.pack=55000.75
        self.loc="Mumbai"
    def p2(self):
        print("Package\t\t: ",self.pack)
        print("Location\t: ", self.loc)
# derived class creation
class D(Person,Personal):
    def m3(self):
        self.cmp="Google"
        print("Company\t\t: ",self.cmp)
# object creation for derived class
obj=D()
print("-------------------------------------")
print("\t Multiple Inheritance")
```

Multiple Inheritance Creation

```
print("--------------------------------------")
# calling methods of base classes and derived class using derived class object
obj.m1()
obj.p1()
obj.m2()
obj.p2()
obj.m3()
```

OUTPUT

```
TERMINAL    JUPYTER    PROBLEMS    OUTPUT    DEBUG CONSOLE

PS C:\Users\Ganesh\Documents\VSC Python Projects\inheritance> py multiple.py
--------------------------------------
         Multiple Inheritance
--------------------------------------
Name            :  Rohit
Id              :  14
Package         :  55000.75
Location        :  Mumbai
Company         :  Google
PS C:\Users\Ganesh\Documents\VSC Python Projects\inheritance>
```

3. MULTILEVEL INHERITANCE

- Creating a new class (sub class) from another sub class or derived class (already inherited class) is called as multilevel inheritance.
- Intermediate class
  - If a class acts as **base class on one side** and acts as **derived class on another side** is called as intermediate class.

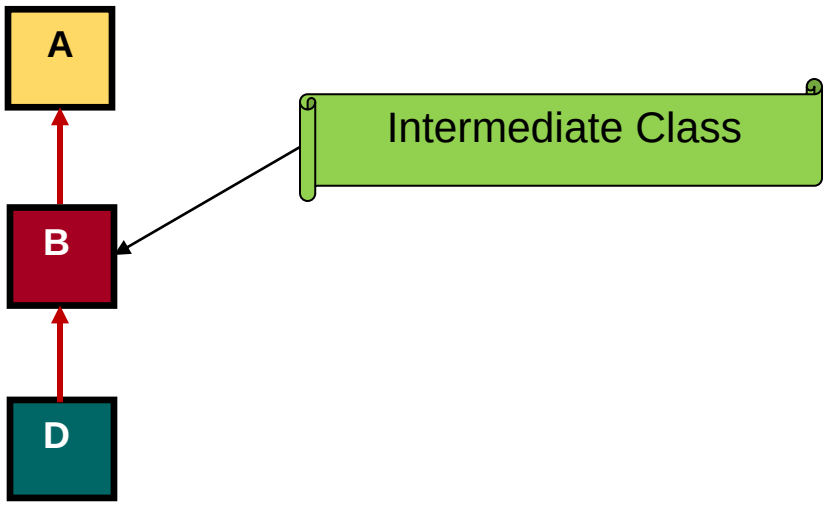


Figure 1.3 Multilevel Inheritance

Where,

- A ← super class
- **B** ← intermediate class
- D ← derived class

3. EXAMPLE OF MULTILEVEL INHERITANCE

(multilevel.py)

SOURCE CODE

```
# base class definition
class A:
    def m1(self):
        self.a=15
# intermediate class definition
class B(A):
    def m2(self):
        self.b=20
# derived class definition
class C(B):
    def m3(sel):
        sel.c=15
    def disp(self):
        self.rs=self.a+self.b+self.c
        print("Input 1: ",self.a)
        print("Input 2: ",self.b)
        print("Input 3: ",self.c)
        print("Sum: ",self.rs)
# object creation for derived class
obj=C()
print("-------------------------------------")
print("\t Multilevel Inheritance")
print("-------------------------------------")
```

Multilevel Inheritance Creation

```
# calling base and derived class methods
obj.m1()
obj.m2()
obj.m3()
obj.disp()
print("-------------------------------------")
```

OUTPUT

```
TERMINAL    JUPYTER    PROBLEMS    OUTPUT    DEBUG CONSOLE

PS C:\Users\Ganesh\Documents\VSC Python Projects\inheritance> py multilevel.py
-------------------------------------
        Multilevel Inheritance
-------------------------------------
Input 1:  15
Input 2:  20
Input 3:  15
Sum:  50
-------------------------------------
PS C:\Users\Ganesh\Documents\VSC Python Projects\inheritance>
```

4. HIERARCHICAL INHERITANCE

- Creating several new classes (sub classes) from **ONLY ONE SUPER CLASS** is called as hierarchical inheritance.
- Like many-1 mapping.

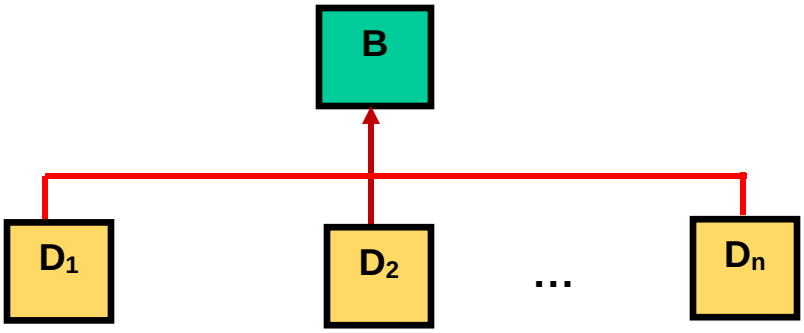


Figure 1.4 Hierarchical Inheritance

Where,

- **D_1, D_2, …D_n** are list of sub classes
- **B** is a super class.

4. EXAMPLE OF HIERARCHICAL INHERITANCE

(hierar.py)

SOURCE CODE

```
# super class definition
class SmartPhone:
    def create(self):
        self.ram="8 GB RAM"
        self.storage="128 GB"
# creation of first derived class
class Oppo(SmartPhone):
    def oppophone(self):
        self.model="Oppo F19 Pro"
        self.color="Silver"
        self.price=19990.00
        print("Model \t\t: ",self.model)
        print("RAM \t\t: ",self.ram)
        print("Internal \t: ",self.storage)
        print("Color \t\t: ",self.color)
        print("Price \t\t: ",self.price)
# creation of second derived class
class Xiaomi(SmartPhone):
    def xiaomi_mobile(self):
        self.model="Xiaomi 11 5G"
        self.color="Pacific Pearl"
        self.price=26999.00
        print("Model \t\t: ",self.model)
        print("RAM \t\t: ",self.ram)
        print("Internal \t: ",self.storage)
        print("Color \t\t: ",self.color)
```

```
        print("Price \t\t: ",self.price)
# creation of third derived class
class Realme(SmartPhone):
    def realme(self):
        self.model="Realme 9 5G"
        self.color="Starry Glow"
        self.price=20999.00
        print("Model \t\t: ",self.model)
        print("RAM \t\t: ",self.ram)
        print("Internal \t: ",self.storage)
        print("Color \t\t: ",self.color)
        print("Price \t\t: ",self.price)
# creation of fourth derived class
class Vivo(SmartPhone):
    def vivophone(self):
        self.model="Vivo Y53S"
        self.color="Deep Sea Blue"
        self.price=15490.00
        print("Model \t\t: ",self.model)
        print("RAM \t\t: ",self.ram)
        print("Internal \t: ",self.storage)
        print("Color \t\t: ",self.color)
        print("Price \t\t: ",self.price)
# object creations for derived classes
print("--------------------------------------")
print("\t Hierarchical Inheritance")
print("--------------------------------------")
# sub class 1 object creation
d1=Oppo()
```

```
d1.create()
d1.oppophone()
print("----------------------------------------")
# sub class 2 object creation
d2=Xiaomi()
d2.create()
d2.xiaomi_mobile()
print("----------------------------------------")
# sub class 3 object creation
d3=Realme()
d3.create()
d3.realme()
print("----------------------------------------")
# sub class 4 object creation
d4=Vivo()
d4.create()
d4.vivophone
print("----------------------------------------")
```

OUTPUT

```
TERMINAL    JUPYTER    PROBLEMS    OUTPUT    DEBUG CONSOLE

PS C:\Users\Ganesh\Documents\VSC Python Projects\inheritance> py hierar.py
---------------------------------------
        Hierarchical Inheritance
---------------------------------------
Model          :  Oppo F19 Pro
RAM            :  8 GB RAM
Internal       :  128 GB
Color          :  Silver
Price          :  19990.0
---------------------------------------
Model          :  Xiaomi 11 5G
RAM            :  8 GB RAM
Internal       :  128 GB
Color          :  Pacific Pearl
Price          :  26999.0
---------------------------------------
Model          :  Realme 9 5G
RAM            :  8 GB RAM
Internal       :  128 GB
Color          :  Starry Glow
Price          :  20999.0
---------------------------------------
Model          :  Vivo Y53S
RAM            :  8 GB RAM
Internal       :  128 GB
Color          :  Deep Sea Blue
Price          :  15490.0
---------------------------------------
PS C:\Users\Ganesh\Documents\VSC Python Projects\inheritance>
```

5. HYBRID INHERITANCE

- Process of creating new class involving **atleast one form of inheritance** (single or multiple or multilevel or hierarchical) and **one or more number of super classes** is called as hybrid inheritance.

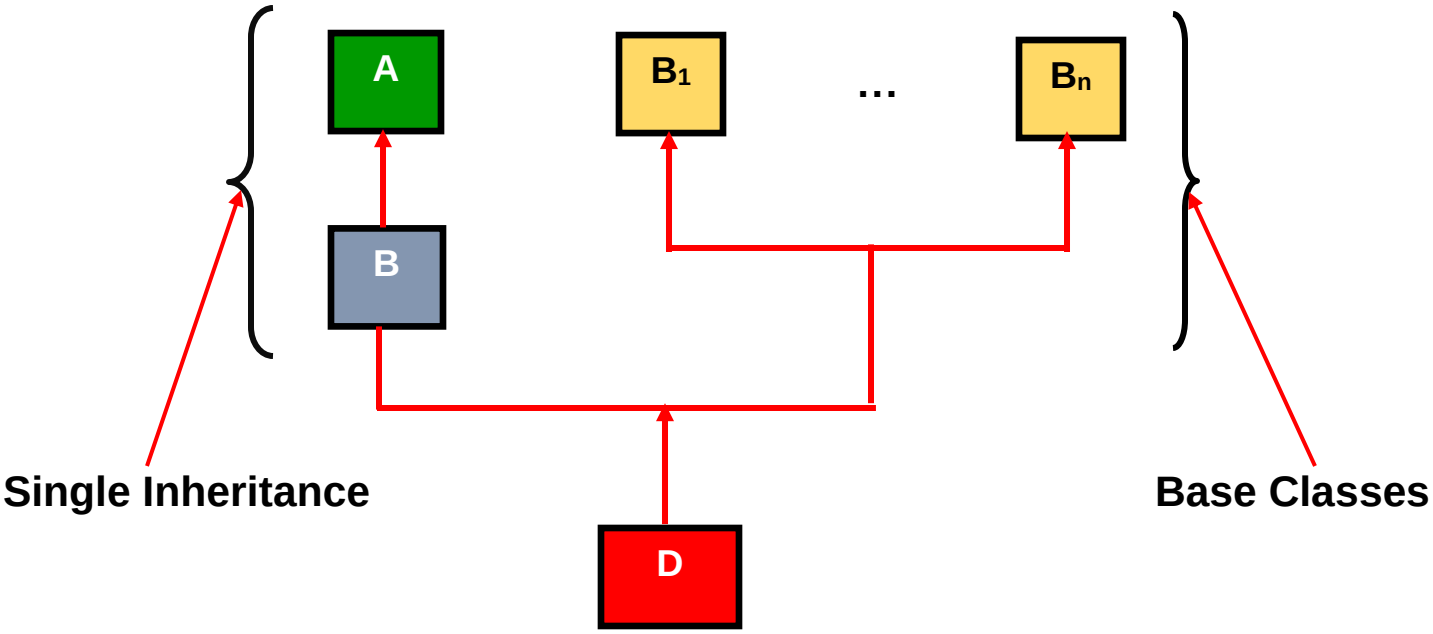


Figure 1.5 Hybrid Inheritance

Where,

- **A→ B is a single inheritance**
- **B_1, … B_n are super classes**
- **D is a sub class.**

5. EXAMPLE OF HYBRID INHERITANCE

(hyb.py)

SOURCE CODE

```
# super class 1
class A:
    def f1(self):
        self.i=5
# single inheritance - sub class
class B(A):
    def f2(self):
        self.j=9
# super class 2
class C:
    def f3(self):
        self.k=15
# super class 3
class D:
    def f4(self):
        self.z=50
# new derived class
class HI(B,C,D):
    def sum4(sel):
        sel.rs=sel.i+sel.j+sel.k+sel.k
        print(" Input 1 \t: ",sel.i)
        print(" Input 2 \t: ",sel.j)
        print(" Input 3 \t: ",sel.k)
        print(" Input 4 \t: ",sel.z)
        print(" Sum \t\t: ",sel.rs)
```

Single Inheritance

Derived Class - Hybrid Inheritance

```
# derived class object creation
obj=HI()
print("-------------------------------------")
print("\t Hybrid Inheritance")
print("-------------------------------------")
# call base class methods and derived class methods using derived class object
obj.f1()
obj.f2()
obj.f3()
obj.f4()
obj.sum4()
```

OUTPUT

```
TERMINAL   JUPYTER   PROBLEMS   OUTPUT   DEBUG CONSOLE

PS C:\Users\Ganesh\Documents\VSC Python Projects\inheritance> py hyb.py
-------------------------------------
        Hybrid Inheritance
-------------------------------------
 Input 1        :  5
 Input 2        :  9
 Input 3        :  15
 Input 4        :  50
 Sum            :  44
PS C:\Users\Ganesh\Documents\VSC Python Projects\inheritance>
```

BENEFITS OF INHERITANCE

- Enhancement of base class
- It saves the storage space and time
- It reduces the code length
- It allows the reusability of code
- It increases the reliability of code.